MemoryAgent `ask` 接口 LOCOMO 优化方案（仅方案，不改功能代码）

0. 关键高亮（颜色标记）

- **【P0】`search_details` 必须先升级为 BM25 + 向量余弦混合检索。**
- **【P0】先冻结 Event Schema v1，再改事件抽取/去重。**
- **【P1】环境字段（`img_url/blip_caption/query/re-download`）需要进入 facts 抽取上下文。**
- **【P1】facts 提取做“两段式抽取（主事实 + 补充事实）”。**
- **【架构建议】Neo4j（KG）+ PostgreSQL(pgvector + FTS)；JSON 降级为缓存/导出。**
- **【面试表达】你不是“纯 JSON”，而是“图谱层 + 检索层 + 编排层”分层架构。**

1. 背景与目标

你在 LOCOMO 测试里提的 6 个点，本质上分成两类：

1. **检索/信息质量问题**
   - 事件定义边界不清
   - `search_details()` 需要从纯向量改为混合检索
   - facts 提取覆盖率仍可提升
   - 对话中混入环境字段时，当前链路信息有损
2. **工程化问题**
   - 需要清晰展示当前 `MemoryAgent.ask` 链路
   - 存储层不能只停在 JSON，本地缓存与线上检索存储要分层

本文只给**落地方案**，不改代码。

---

2. 当前 `MemoryAgent.ask` 真实链路（基于当前仓库）

2.1 在线问答链路（Ask）

入口：

- `src/m_agent/agents/memory_agent.py`
  - `ask()` -> `deep_recall()`

主流程：

1. **策略判断**
   - `_detect_direct_answer_strategy()` 判断是否先直答。
2. **直答路径**
   - `_answer_directly()` 调工具代理直接检索；
   - 若直答证据不足，fallback 到拆解路径。
3. **拆解路径**
   - `_decompose_question()` 生成子问题计划；

- `_solve_sub_questions()` 执行子问题；
- `_synthesize_final_answer()` 汇总答案。

4. **统一收尾**
   - `_finalize_recall_payload()` 输出 `answer/gold_answer/evidence/tool_calls/question_plan`。

可用工具（`_build_tools`）：

- `resolve_entity_id`
- `get_entity_profile`
- `search_entity_feature`
- `search_entity_event`
- `search_entity_events_by_time`
- `search_content`
- `search_events_by_time_range`
- `search_details`

当前 prompt 策略已倾向普通细节优先 `search_details`（`config/agents/memory/runtime/agent_runtime.yaml`）。

2.2 数据构建链路（离线/导入）

导入主线：

- `src/m_agent/memory/memory_core/workflow/build/load_from_episode.py`
  1. `scan_and_form_scenes`
  2. `scan_and_form_scene_facts`
  3. `extract_fact_entities`
  4. `import_fact_entities`

facts 生成核心：

- `src/m_agent/memory/build_memory/form_scene_details.py`
  - 对 episode 做 LLM facts 抽取
  - 失败时 fallback keyword 抽取
  - 为 Atomic fact 生成 embedding
  - 去重并写回 `scene/*.json` 的 `facts`

2.3 当前 `search_details` 检索方式

- `src/m_agent/memory/memory_core/workflow/search/details_search.py`
- 当前是：**对 Atomic fact embedding 做纯余弦相似度检索**
- 无 BM25 / 关键词倒排 / 稀疏检索融合

---

3. 你提的 6 个问题与根因映射

3.1 事件定义不清

现状：

- 事件是否可用先走 `facts_filter`（`config/prompts/filtering/facts_filter.yaml`）。
- 事件抽取走 `event_extract_prompt`（`config/memory/core/runtime/memory_core_runtime.yaml`）。

根因：

- “event”语义边界仍偏宽泛（动作、状态、计划、结果混在一起）。
- 去重时对“同一事件不同阶段”区分不够强。

3.2 `search_details` 必须改混合检索

现状是纯 dense，所以会出现：

- 实体名、时间词、稀有关键词命中不稳定；
- 语义接近但关键字不符时更容易误召回。

3.3 facts 提取覆盖率

现状其实不差（你的判断成立），但仍有改进空间：

- 长 episode 信息拥挤时可能漏细节；
- 单轮输出对“上下文条件/因果补充”覆盖不稳定。

3.4 原始对话含环境字段时信息丢失

LOCOMO 数据中 message 除 `speaker/text/dia_id` 外，还可能有：

- `img_url`
- `blip_caption`
- `query`
- `re-download`

当前 `locomo_history_loader.py` 只保留 `speaker/text/timestamp/turn_id`，环境字段被丢弃，后续 facts 抽取不可见。

3.5 LangChain“太简单”问题

现状是“**轻 LangChain 编排 + 自研 memory core 工作流**”：

- 优点：可控、易调优、链路透明；
- 缺点：抽象层次偏低，缺统一可视化编排与标准化检索组件复用。

3.6 存储层“纯 JSON”面试风险

你现在并非完全无数据库：

- KG 已有 Neo4j（`kg_base.py` / `neo4j_store.py`）。

但面试官会质疑是合理的，因为在线细节检索仍主要依赖本地 `scene/*.json` 扫描，工程感不足。

---

4. 分阶段改造方案（建议）

Phase 0（最高优先级，先打 LOCOMO 分数）

A. 统一事件定义（先定规范，再做代码）

先冻结一个 **Event Schema v1**（文档级规范）：

- `event_core`：事件核心谓词（必须）
- `event_type`：action/state/plan/decision/result（必须）
- `participants`：主体/客体（可选）
- `time_actual`：绝对时间段（可空）
- `time_abstract`：频率/习惯（可空）
- `context`：地点/条件/原因（可选）
- `evidence_ref`：`dialogue_id/episode_id/turn_span`

规则重点：

- “说话行为（said/replied）默认不算事件”，除非“说这件事本身就是关键事件”。
- “状态”和“动作”分开；“计划”和“已发生事件”分开。
- 同事件多阶段（计划->执行->结果）保留为不同事件，靠 `event_type + time` 区分。

B. `search_details` 改为混合检索（BM25 + Cosine）

目标：

- Lexical 负责“关键词/实体词/时间词”精确召回
- Dense 负责“语义泛化”召回

建议检索协议：

1. **候选召回**
   - 稀疏：BM25(topN_sparse)
   - 稠密：Cosine(topN_dense)
2. **融合排序**
   - 默认 RRF（稳健，易调）
   - 备选：线性融合 `score = w_dense*dense_norm + w_sparse*bm25_norm`
3. **输出**
   - 返回 fused score + 子分数，便于调参与诊断

初始参数（建议起点）：

- `topN_sparse=30`
- `topN_dense=30`
- `topk_final=5~10`
- RRF `k=60`（或线性 `w_dense=0.6, w_sparse=0.4` 起步）

C. 评估与回归

建立 LOCOMO 小集回归（先 50~100 题）：

- `Recall@k`
- `MRR`
- 答案 EM/F1
- 失败样本分类（漏召回 / 误召回 / 时间错位 / 实体错位）

Phase 1（次优先级）

A. facts 提取覆盖率增强

用"两段式抽取"替代单段式：

1. 主抽取：核心 Atomic facts
2. 补充抽取：时间、条件、因果、对象等可检索补充事实

再做统一去重合并（保留 evidence 粒度）。

B. 环境字段接入

在 loader 与 facts 抽取输入中保留环境字段：

- turn 结构新增 `env`（可选 dict）
- 抽取 prompt 输入中拼入结构化环境块

建议链路：

- `原始turn(文本+env)` -> `规范化复述(可选)` -> `facts抽取`

注意：复述只作为中间特征，不替代原文证据；最终 evidence 仍要可追溯到原 turn。

Phase 2（工程化与面试可讲）

5. 存储与框架选型建议

5.1 结论先行（推荐）

推荐架构：Neo4j + PostgreSQL(pgvector + FTS) 双存储

- Neo4j：继续承载实体关系/KG
- PostgreSQL：承载原始记忆、facts、向量、全文索引与混合检索
- JSON：降级为缓存/导出，不再是主检索存储

这个组合最适合你当前形态：

- 已有 Neo4j 不浪费
- Postgres 改造成本可控，面试叙事强
- 可一库完成 BM25(FTS) + vector + metadata filter + 事务治理

5.2 备选方案

1. **OpenSearch**
   - 优点：原生混合检索、查询能力强、工业成熟
   - 缺点：运维和资源成本通常高于 Postgres

2. **Qdrant / Weaviate / Pinecone**
    - 优点：向量与混合检索开箱快
    - 缺点：通常仍需外部事务型库承载业务主数据

5.3 LlamaIndex/LangChain定位

- **LlamaIndex、LangChain是编排框架，不是“必须绑定的数据库”。**
- LlamaIndex 支持大量 vector store 集成，适合做适配层与快速实验；
- 你当前更需要先把“数据模型 + 检索质量 + 存储分层”打稳，再考虑是否切更重的框架编排。

---

6. 实施路线（建议 4 周）

1. **第 1 周**
    - 冻结 Event Schema v1
    - 设计混合检索评分协议与评测集
2. **第 2 周**
    - `search_details` 混合检索 PoC
    - 输出可解释 score（dense/sparse/fused）
3. **第 3 周**
    - 环境字段接入 + facts 两段式抽取 PoC
    - LOCOMO 回归对比
4. **第 4 周**
    - Postgres 存储落地（最小可用）
    - JSON 角色降级、迁移脚本与面试文档沉淀

---

7. 面试沟通版本（可直接说）

1. 我们把记忆系统拆成了三层：
    - **KG层（Neo4j）** 做实体关系
    - **检索层（Postgres + pgvector + FTS）** 做混合召回
    - **编排层（MemoryAgent/LangChain）** 做问题分解和工具路由
2. `ask` 默认走细节检索，复杂问题再拆解，降低过度规划开销。
3. `search_details` 从纯向量升级到 BM25+向量融合，显著降低关键词漏召回。
4. JSON 只做离线缓存和可观测导出，不再作为线上主检索存储。

---

8. 当前阶段建议的优先级（与你的判断一致）

1. `search_details` 混合检索（最高）
2. 事件定义收敛（最高）
3. 环境字段接入（中高）
4. facts 提取增强（中）
5. 存储工程化（中高，和 1/2 可并行设计）